

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ СИСТЕМ УПРАВЛЕНИЯ И РОБОТОТЕХНИКИ

РАЗРАБОТКА СИМУЛЯЦИОННОЙ СИСТЕМЫ ПОИСКА
И ПОСАДКИ БЕСПИЛОТНОГО ЛЕТАТЕЛЬНОГО АППАРАТА
НА ARUCO-МАРКЕР В ROS 2 И GAZEBO

Аннотация – В работе разработана симуляционная система автономного квадрокоптера на базе ROS 2 Jazzy и Gazebo Sim для поиска и посадки по ArUco-маркеру. Система включает модель дрона с нижней камерой и 2D-лидаром, миры Gazebo, ROS-Gazebo bridge, визуализацию RViz и узлы Python управления. Реализованы два режима миссии: зигзагообразное сканирование открытой арены и полет в лабиринте с реактивным обходом препятствий. В обоих сценариях при обнаружении маркера дрон прерывает поиск, центрирует его в кадре и выполняет управляемое снижение до посадки.

Ключевые слова: ROS 2 Jazzy, Gazebo Sim, drone, ArUco, OpenCV, visual servoing, lidar, docking, autonomous landing, Docker.

Выполнили

Май Хоанг Куан

Ису: 407882,

*ТГ: @quanmh_25,
itmoquanmh@gmail.com*

Проверил

Живицкий Андрей Юрьевич

Содержание

1	Введение	2
2	Структура проекта	2
2.1	Пакет <code>drone_scene</code>	3
2.2	Пакет <code>drone_control</code>	3
3	Симуляционная модель дрона	3
3.1	Нижняя камера	3
3.2	Лидар	4
3.3	Плагины Gazebo	4
4	ROS 2 интерфейсы	4
5	ArUco-детекция	5
6	Режим покрывающего поиска	6
6.1	Зигзагообразная траектория	6
6.2	Переход к посадке	6
7	Режим лабиринта	6
7.1	Состояния миссии	6
7.2	Обход препятствий по лидару	7
8	Алгоритм посадки	7
8.1	Центрирование маркера	7
8.2	Снижение	8
9	Запуск проекта	8
9.1	Запуск через Docker	8
9.2	Нативный запуск	8
10	Результаты	8
11	Особенности реализации	9
12	Ограничения и возможные улучшения	10
13	Заключение	10

1 Введение

Автономная посадка беспилотного летательного аппарата на визуально размеченную площадку является типовой задачей мобильной робототехники. Такая задача объединяет несколько важных компонентов: симуляцию физического окружения, получение сенсорных данных, компьютерное зрение, управление движением и организацию программной архитектуры в ROS 2.

Цель проекта состоит в создании симуляционной системы, в которой дрон способен:

- взлетать и выполнять поиск посадочной площадки;
- получать изображение с нижней камеры;
- обнаруживать ArUco-маркеры с помощью OpenCV;
- отличать целевой маркер посадки от дополнительных маркеров сцены;
- в открытой среде выполнять покрывающее движение по карте;
- в лабиринте использовать 2D-лидар для обхода стен;
- при обнаружении ArUco ID 50 переходить из режима поиска в режим посадки;
- центрировать маркер в кадре и плавно снижаться до земли;
- запускаться как нативно в ROS 2 workspace, так и через Docker.

Проект оформлен как ROS 2 workspace и состоит из двух основных пакетов: `drone_scene` и `drone_control`. Первый отвечает за симуляционную сцену, модель дрона, миры Gazebo и мосты сообщений. Второй содержит управляющие ROS 2 узлы, детектор ArUco и launch-файлы миссий.

2 Структура проекта

Основная структура репозитория представлена в таблице 1. Такое разделение делает проект удобным для проверки: симуляция и алгоритмы управления находятся в разных пакетах, а запуск миссий выполняется через отдельные launch-файлы.

Путь	Назначение
<code>workspace/src/drone_scene</code>	Gazebo-миры, модель дрона, Xacro/URDF-описание, RViz-конфигурация, модели ArUco-маркеров и конфигурация <code>ros_gz_bridge</code> .
<code>workspace/src/drone_control</code>	ROS 2 Python-пакет с управляющими узлами, ArUco-детектором и launch-файлами для режимов <code>coverage</code> и <code>maze</code> .
<code>docs/</code>	Демонстрационный видеоролик и изображение-превью результата работы.
<code>Dockerfile</code>	Описание Docker-образа с ROS 2 Jazzy, Gazebo, RViz, OpenCV, <code>cv_bridge</code> и зависимостями проекта.
<code>docker-compose.yaml</code>	Однокомандный запуск проекта с графическими окнами Gazebo и RViz.

Таблица 1: Структура репозитория

2.1 Пакет `drone_scene`

Пакет `drone_scene` реализует симуляционную часть проекта. В нем находятся два мира:

- `arena.sdf` — открытая площадка для покрывающего поиска;
- `maze.sdf` — лабиринт для проверки полета с обходом препятствий.

Также пакет содержит модель дрона `drone.xacro`. В модели определены корпус, визуальные mesh-объекты, нижняя камера, 2D-лидар, плагин управления скоростью и плагин публикации одометрии.

2.2 Пакет `drone_control`

Пакет `drone_control` содержит три исполняемых узла:

- `coverage_node` — тестовый узел движения по покрывающей траектории;
- `coverage_docking_node` — миссия открытой арены с поиском ArUco и посадкой;
- `maze_docking_node` — миссия лабиринта с лидаром, обходом препятствий и посадкой.

Общий модуль `aruco_detector.py` инкапсулирует работу с OpenCV ArUco: создание словаря, настройку параметров детектора, поиск углов маркеров и, при необходимости, оценку позы одиночного маркера.

3 Симуляционная модель дрона

Модель дрона описана в файле:

```
workspace/src/drone_scene/urdf/drone.xacro
```

Основная ссылка модели — `base_link`. Для визуального отображения используются mesh-файлы корпуса и четырех пропеллеров. Для физического взаимодействия задана простая box-collision геометрия и инерциальные параметры.

3.1 Нижняя камера

Камера закреплена на звене `downward_camera_link` через фиксированное соединение `camera_joint`. Она направлена вниз и используется для поиска маркеров на полу. Основные параметры камеры:

- разрешение изображения: 640×480 ;
- частота обновления: 30 Гц;
- топик Gazebo/ROS: `/camera/image_raw`;
- дополнительный топик калибровки: `/camera/camera_info`.

В управляющих узлах используется приближенная матрица камеры:

$$K = \begin{bmatrix} 530 & 0 & 320 \\ 0 & 530 & 240 \\ 0 & 0 & 1 \end{bmatrix}.$$

Центр изображения принимается равным (320, 240) пикселей. Это позволяет использовать простое визуальное сервоуправление: если центр маркера смещен относительно центра изображения, дрон получает команды по осям x и y , чтобы вернуть маркер в центр кадра.

3.2 Лидар

Для режима лабиринта в модели добавлен `gpu_lidar`, закрепленный на звене `lidar_link`. Лидар публикует данные в топик `/scan`. В текущей реализации он используется как реактивный датчик препятствий: управляющий узел анализирует минимальные расстояния в переднем, левом и правом секторах и выбирает направление поворота при встрече со стеной.

3.3 Плагины Gazebo

Для движения используется плагин:

`gz-sim-velocity-control-system`

Он принимает команды `geometry_msgs/msg/Twist` через топик `/cmd_vel`. Одометрия публикуется плагином:

`gz-sim-odometry-publisher-system`

Одометрия доступна в ROS 2 через топик `/odom`. Параметр `dimensions=3` позволяет использовать трехмерное положение дрона, включая высоту z .

4 ROS 2 интерфейсы

Связь между Gazebo и ROS 2 организована через файл:

`workspace/src/drone_scene/config/ros_gz_bridge.yaml`

Ключевые топики проекта перечислены в таблице 2.

Таблица 2: Основные ROS 2 топики проекта

Топик	Тип сообщения	Назначение
<code>/camera/image_raw</code>	<code>sensor_msgs/msg/Image</code>	Изображение с нижней камеры дрона.
<code>/camera/camera_info</code>	<code>sensor_msgs/msg/CameraInfo</code>	Параметры камеры.

Топик	Тип сообщения	Назначение
/cmd_vel	geometry_msgs/msg/Twist	Команды линейной и угловой скорости дрона.
/odom	nav_msgs/msg/Odometry	Положение и ориентация дрона в симуляции.
/scan	sensor_msgs/msg/LaserScan	Данные 2D-лидара для обнаружения стен и препятствий.

5 ArUco-детекция

Для поиска маркеров используется OpenCV ArUco. Детектор создается в классе `ArucoMarkerDetector`. В проекте выбран словарь:

`cv2.aruco.DICT_4X4_250`

Алгоритм обработки изображения состоит из следующих шагов:

1. ROS-сообщение `sensor_msgs/msg/Image` преобразуется в OpenCV-изображение через `cv_bridge`.
2. Функция `cv2.aruco.detectMarkers` находит углы и идентификаторы маркеров.
3. Для каждого найденного ID проверяется его смысл в текущей миссии.
4. Если обнаружен ID 50, вычисляется центр маркера как среднее значение координат четырех углов.
5. Ошибка центрирования определяется как разность между центром маркера и центром изображения:

$$e = \begin{bmatrix} e_u \\ e_v \end{bmatrix} = \begin{bmatrix} u_{marker} - 320 \\ v_{marker} - 240 \end{bmatrix}.$$

6. Ошибка используется для формирования команд перемещения дрона над посадочной площадкой.

В сценах используются несколько маркеров. Целевой маркер посадки — ID 3. Остальные маркеры служат дополнительными объектами окружения:

- ID 3 — посадочная площадка;
- ID 42 — маркер угрозы, на него посадка не выполняется;
- ID 23 — маркер груза или точки снабжения;
- ID 50 и ID 65 — дополнительные маркеры сцены.

6 Режим покрывающего поиска

Режим открытой арены запускается командой:

```
1 ros2 launch drone_control coverage_run.launch.py
```

Этот launch-файл подключает сцену `coverage_sim.launch.py`, которая открывает мир `arena.sdf`, запускает Gazebo, RViz, `robot_state_publisher`, создает модель дрона и включает мост Gazebo–ROS. Через задержку 7 секунд запускается узел `coverage_docking_node`.

6.1 Зигзагообразная траектория

В открытой арене дрон должен просканировать область размером примерно 8×8 м. Для этого генерируется набор waypoint-точек:

- границы по x : от -4 до 4 м;
- границы по y : от -4 до 4 м;
- шаг между проходами: 1.5 м;
- высота полета: $z = 2.0$ м.

Дрон последовательно летит от одной точки к другой, формируя покрывающий зигзаг. Управление положением реализовано как пропорциональный регулятор:

$$v_x = k_{xy}(x_{target} - x), \quad v_y = k_{xy}(y_{target} - y), \quad v_z = k_z(z_{target} - z).$$

В коде используются коэффициенты $k_{xy} = 0.5$ и $k_z = 0.8$, а скорости ограничиваются по модулю, чтобы движение оставалось плавным и устойчивым.

6.2 Переход к посадке

Пока дрон находится в состоянии **SEARCHING**, он продолжает движение по waypoint-точкам. При обнаружении ArUco ID 3 узел сохраняет ошибку центрирования и переводит автомат в состояние **ALIGNING**. После этого покрывающий поиск прекращается, так как найдена посадочная площадка.

7 Режим лабиринта

Режим лабиринта запускается командой:

```
1 ros2 launch drone_control maze_run.launch.py
```

Этот режим использует мир `maze.sdf` и узел `maze_docking_node`. В отличие от открытой арены, здесь дрон не летит по заранее заданным waypoint-точкам, а выполняет реактивное движение вперед с обходом стен.

7.1 Состояния миссии

Главный автомат узла `maze_docking_node` содержит следующие состояния:

- **TAKEOFF** — набор высоты до стабильного уровня;
- **SEARCHING** — движение по лабиринту и поиск маркера;

- **ALIGNING** — центрирование ArUco ID 50 в кадре;
- **DESCENT** — снижение с сохранением центрирования;
- **LANDED** — остановка после посадки.

Высота полета в лабиринте задается параметром:

$$z_{hover} = 1.6 \text{ м.}$$

В состоянии **TAKEOFF** дрон поднимается до этой высоты. После достижения допуска по высоте узел переходит в **SEARCHING**.

7.2 Обход препятствий по лидару

Для обхода препятствий лидарное сканирование делится на три сектора:

- передний сектор: от -20° до 20° ;
- левый сектор: от 60° до 120° ;
- правый сектор: от -120° до -60° .

Если минимальная дистанция в переднем секторе становится меньше `stop_dist = 0.3` м, дрон переходит во внутренний режим **AVOID**. Направление поворота выбирается по более свободной стороне: если слева расстояние больше, дрон поворачивает влево, иначе вправо. Когда передний сектор очищается больше чем на `clear_dist = 0.5` м, дрон возвращается к движению вперед.

Такой подход не строит глобальную карту лабиринта, но хорошо подходит для простой демонстрации автономного поиска: дрон поддерживает высоту, идет вперед, реагирует на ближайшие стены и продолжает искать визуальный маркер.

8 Алгоритм посадки

Обе миссии используют одинаковую идею посадки: сначала дрон ищет целевой маркер, затем выравнивается над ним и только после этого начинает снижение.

8.1 Центрирование маркера

В состоянии **ALIGNING** управляющий узел использует пиксельную ошибку:

$$e_u = u_{marker} - 320, \quad e_v = v_{marker} - 240.$$

Так как камера направлена вниз, смещение маркера в изображении преобразуется в горизонтальное движение дрона:

$$v_x = -k_{px}e_v, \quad v_y = -k_{py}e_u.$$

В проекте используется коэффициент:

$$k_{px} = 0.002.$$

Скорости ограничиваются диапазоном примерно от -0.2 до 0.2 м/с. Это не дает дрону резко ускоряться при большом смещении маркера. Центрирование считается успешным, если:

$$|e_u| < 25, \quad |e_v| < 25.$$

8.2 Снижение

После центрирования дрон переходит в состояние DESCENT. В этом состоянии он продолжает мягко корректировать горизонтальное положение по маркеру и одновременно снижает высоту. В режиме открытой арены используется постоянная скорость снижения около -0.18 м/с. В режиме лабиринта скорость снижения зависит от текущей высоты: выше дрон снижается быстрее, у земли медленнее.

Посадка считается завершенной, когда высота по одометрии становится меньше 0.18 м. После этого состояние меняется на LANDED, а в `/cmd_vel` публикуется нулевая команда.

9 Запуск проекта

9.1 Запуск через Docker

Для удобства проверки подготовлен Docker workflow. Сначала нужно разрешить контейнеру открывать графические окна:

```
1 xhost +local:docker
```

Запуск миссии лабиринта по умолчанию:

```
1 docker compose up
```

Запуск покрывающей миссии:

```
1 LAUNCH_FILE=coverage_run.launch.py docker compose up
```

В `docker-compose.yaml` используется опубликованный образ:

```
quanmh25/drone_aruco_docking:latest
```

Контейнер монтирует текущий репозиторий, собирает workspace командой `colcon build --symlink-install`, подключает `install/setup.bash` и запускает выбранный launch-файл.

9.2 Нативный запуск

Для нативного запуска требуется ROS 2 Jazzy и пакеты Gazebo/ROS. Сборка выполняется из директории `workspace`:

```
1 cd workspace
2 source /opt/ros/jazzy/setup.bash
3 colcon build --symlink-install
4 source install/setup.bash
```

Команды запуска:

```
1 ros2 launch drone_control maze_run.launch.py
2 ros2 launch drone_control coverage_run.launch.py
```

10 Результаты

В результате работы был создан проект, который демонстрирует полный цикл автономной миссии дрона в симуляции:

1. запуск Gazebo и RViz;
2. появление модели дрона в выбранном мире;

3. публикация изображения камеры, одометрии, лидара и команд скорости;
4. поиск ArUco-маркера в изображении нижней камеры;
5. различение целевого маркера ID 50 и дополнительных маркеров;
6. переход из режима поиска в режим визуального центрирования;
7. снижение и остановка после посадки.

Демонстрационный результат сохранен в директории `docs/`. На рисунке 1 показан кадр демонстрации.

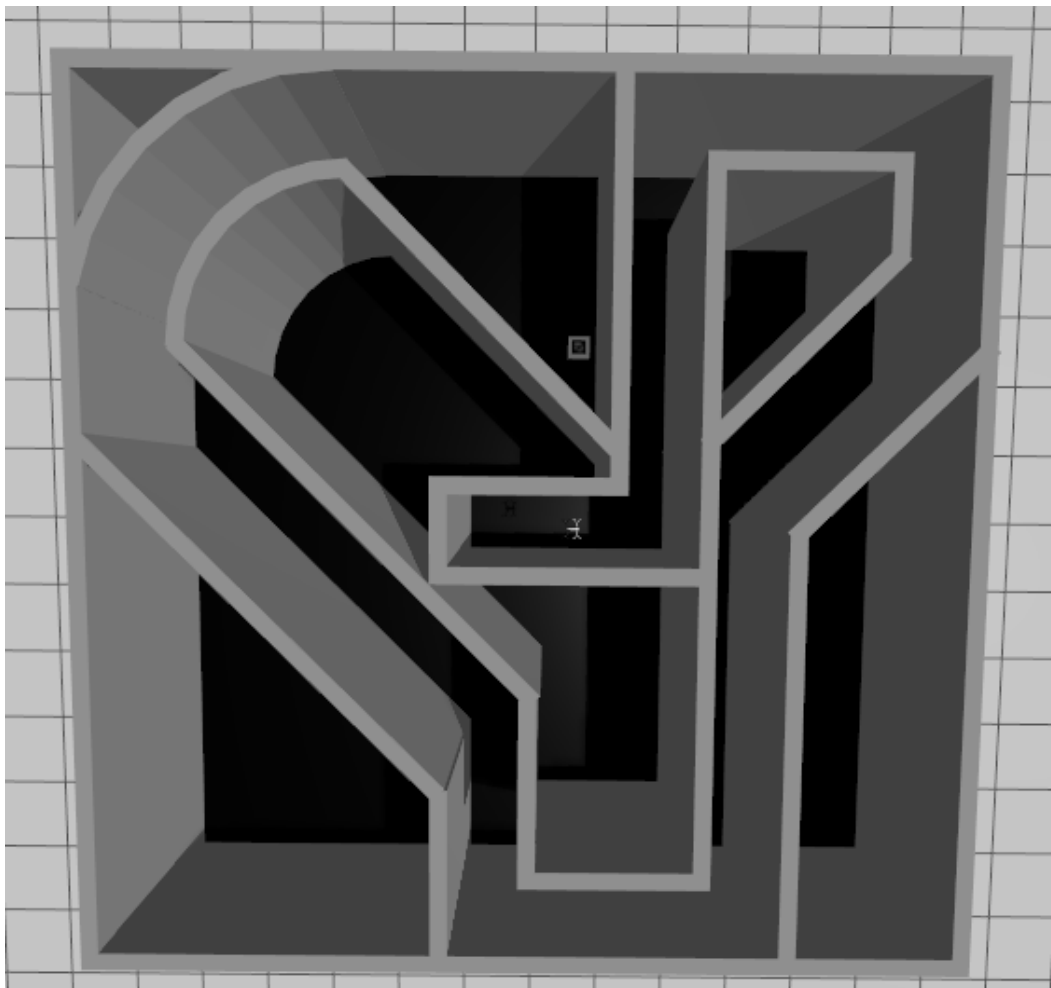


Рис. 1: Демонстрационный кадр работы проекта: дрон выполняет поиск и посадку на ArUco-площадку в Gazebo.

11 Особенности реализации

Главная особенность проекта состоит в том, что поиск и посадка оформлены как конечный автомат. Это делает поведение дрона понятным и расширяемым: каждое состояние отвечает за один этап миссии, а переходы происходят по конкретным условиям, например по достижению высоты, обнаружению маркера или успешному центрированию.

Вторая важная особенность — наличие двух режимов работы. Открытая арена проверяет способность дрона систематически покрывать область поиска. Лабиринт проверяет

более сложный сценарий, где нужно не только искать маркер, но и реагировать на препятствия по данным лидара.

Третья особенность — воспроизводимость запуска. Проект можно запустить нативно в ROS 2 workspace или через Docker Compose. Это уменьшает зависимость от локальной конфигурации системы проверяющего и упрощает демонстрацию.

12 Ограничения и возможные улучшения

Текущая реализация решает основную задачу поиска и посадки, однако имеет ряд ограничений:

- используется простая пиксельная ошибка маркера, без полноценной 3D-оценки относительной позы для управления;
- реактивный обход препятствий в лабиринте не строит карту и не гарантирует оптимальный маршрут;
- параметры регуляторов подобраны вручную под текущую симуляцию;
- при длительной потере маркера во время посадки можно добавить более сложную стратегию повторного поиска;
- модель управления дрона упрощена и использует velocity-control plugin, а не полноценную динамику квадрокоптера с отдельными моторами.

В дальнейшем проект можно расширить следующим образом:

- использовать `estimatePoseSingleMarkers` для вычисления 3D-положения маркера относительно камеры;
- добавить PID-регулятор высоты и горизонтального положения;
- реализовать сохранение карты или frontier-based exploration для более сложного поиска;
- добавить обработку временной потери маркера с возвратом к последней надежной позиции;
- расширить RViz-визуализацию траектории, состояния автомата и найденных маркеров.

13 Заключение

В ходе работы была разработана симуляционная система автономного поиска и посадки дрона на ArUco-маркер. Проект объединяет ROS 2 Jazzy, Gazebo Sim, OpenCV ArUco, ROS-Gazebo bridge, RViz и Docker. Были реализованы два сценария миссии: покрывающее сканирование открытой арены и полет в лабиринте с обходом препятствий по лидару.

Целевой сценарий работы выглядит следующим образом: дрон запускается в Gazebo, получает данные камеры и одометрии, выполняет поиск, обнаруживает ArUco ID 50, центрирует маркер в изображении нижней камеры, снижает высоту и завершает посадку. Таким образом, проект демонстрирует практическое применение визуального сервоуправления и ROS 2-архитектуры для задачи автономной посадки беспилотного летательного аппарата.